Proof of Invariant (PoI): Topological Guardians, Equivocation Tracking, and Deterministic Convergence for Causal DAG Ledgers

Oleksandr A. Shvets
Ugoda Protocol Core Research

Draft October 18, 2026

Abstract

Distributed ledgers based on Directed Acyclic Graphs (Causal DAGs) offer high concurrent transaction throughput but face structural vulnerabilities, including double-spending via branch divergence, validator equivocation, and proposer-driven ordering manipulation. In this paper, we analyze Proof of Invariant (PoI), a verification framework that decouples topological graph admissibility from state execution. We show that directed cycles are prevented by content-addressed parent references and verified inductive round-height progression under preimage resistance. We introduce an exact $O(1)$ sequence-indexed equivocation guard with committed-head pruning. Furthermore, we define a deterministic state execution model $S = \text{Execute}(\text{Canonicalize}(D), S_0)$ protected against proposer grinding by round-indexed future-beacon salting, and establish conditional finality using quorum-signed checkpoint certificates over committed anchors.

1 Introduction and Problem Statement

Causal DAG ledgers allow transactions to reference multiple prior heads concurrently, forming merge structures (e.g., diamond topologies $A \to B$, $A \to C$, $B \to D$, $C \to D$). However, naïve DAG validation and consensus designs encounter several systemic challenges:

- **Equivocation tracking.** Naïve graph heuristics fail to detect multi-branch double-signing by validators unless tracking mechanisms enforce strict sequence and self-parent accountability.
- **Proposer grinding and ordering manipulation.** Unsalted lexical sorting of transaction identifiers invites hash grinding by proposers seeking ordering priority.
- **Reorganization and prefix stability.** Unbounded causal histories allow late-arriving deltas to trigger reorganizations unless finality gadgets freeze historical execution prefixes.
- **State execution divergence.** Asynchronous message delivery requires a strict, globally identical linear ordering rule to ensure all nodes execute concurrent state transitions identically.

Proof of Invariant (PoI) addresses these vulnerabilities through five architectural components:

- **Inductive topological safety.** Content-addressed parent references combined with verified self-chain rules and round-height progression guarantee directed acyclic structure.
- **Sequence-based equivocation tracking.** An $O(1)$ in-memory index keyed by validator identity and sequence number detects double-signing, supported by committed-head pruning.

- **Anchor-based segments and future-beacon salting.** Round-based anchor commitments and per-round future beacons mitigate hash grinding and give identical linear execution across nodes.
- **Deterministic conflict resolution.** Double-spends are serialized via canonical segment ordering; conflicting later transactions result in explicit execution reverts (`STATUS_REVERTED`).
- **Conditional finality and committed-anchor checkpoints.** Quorum-signed checkpoints commit to deterministic anchor states, freezing historical execution prefixes.

2 System Model, Delta Format and Structural Invariants

Definition 1 (Causal delta and content addressability)**.** *Let a ledger history be a directed acyclic graph $G = (V, E)$. A delta $d \in V$ is the tuple*

$$d = \langle \text{ChainID}, r, h, P(d), \text{seq}, \text{payload}, \sigma_k \rangle,$$

where r is the verified round number, h is the causal height, $P(d) \subset V$ is the set of parent identifiers, seq *is a per-validator monotonic sequence number, and* $\text{DeltaID}(d) = H(d)$ *is a collision-resistant cryptographic hash over the contents of d.*

Verified delta rules. Nodes recompute and verify all structural fields rather than trusting creator-supplied metadata.

- **Self-chain rule.** $P(d)$ contains exactly one self-parent $s(d)$ (the signer's previous delta), with $\text{seq}(d) = \text{seq}(s(d)) + 1$ and $r(d) > r(s(d))$. Each validator thus produces at most one delta per round and maintains a verifiable linear chain.
- **Round rule.** $r(d) = 1 + \max_{p \in P(d)} r(p)$, and $P(d)$ includes deltas from at least Q distinct signers at round $r(d) - 1$. An honest validator references every parent-closed round-$(r-1)$ delta it holds after waiting a timeout τ.
- **Height rule.** $h(d) = 1 + \max_{p \in P(d)} h(p)$.
- **Quorum bound.** The unweighted quorum threshold is

$$Q = \left\lceil \tfrac{N+f+1}{2} \right\rceil \implies 2Q - N \geq f + 1.$$

  For $N = 3f + 1$ this yields $Q = 2f + 1$.

Lemma 1 (Inductive acyclicity via verified parent closure and self-chain)**.** *If every accepted delta d satisfies verified parent closure ($\forall p \in P(d),\ p \in V_{\text{committed}}$) and verified round-height progression, then $G = (V, E)$ is acyclic.*

Proof. By induction on commit order. The genesis anchor has no parents and is the base case ($r = 0,\ h = 0$). A subsequent delta d is included only after all $p \in P(d)$ are committed. Since $r(d) > r(p)$ and $h(d) > h(p)$ for all parents, commit order is a strict topological order. A directed cycle $d_k \to \cdots \to d_0 \to d_k$ would require d_k to be committed before itself, which is impossible under inductive commit ordering and preimage resistance of content-addressed hashes. □

3 Sequence-Based Equivocation Guard and Pruning

Graph connectivity structures such as Disjoint Set Union cannot detect validator equivocation, because concurrent deltas in a healthy DAG lie in the same weakly connected component. We use an exact sequence-indexed tracker instead.

Definition 2 (Validator equivocation)**.** *A validator v_i has equivocated if there exist two distinct deltas d_a, d_b signed by v_i with*

$$\text{Signer}(d_a) = \text{Signer}(d_b) = v_i, \quad \text{seq}(d_a) = \text{seq}(d_b), \quad \text{DeltaID}(d_a) \neq \text{DeltaID}(d_b).$$

Nodes maintain an in-memory index

$$\mathcal{I} : (\text{ValidatorID}, \text{seq}) \to \text{DeltaID}.$$

When a valid delta arrives, the node verifies its recomputed fields and checks $\mathcal{I}(v_i, \text{seq})$. If the entry is empty it is set to $\text{DeltaID}(d)$. If it already holds a different identifier, the node stores both signed deltas as cryptographic evidence of equivocation and triggers role ejection.

Committed-head pruning. To bound memory, nodes keep only the committed head $(\text{seq}_c, \text{DeltaID}_c)$ of each validator at the latest checkpoint. Any later delta with $\text{seq} \leq \text{seq}_c$ that is not part of the committed history is itself evidence of equivocation, so older index entries can be pruned without losing detection.

4 Rounds, Anchors, Segments and Future-Beacon Salting

4.1 Slots and anchor candidates

The anchor candidate of round r is designated by $\text{slot}(r) = r \bmod N$. Because a round cannot exist until Q validators have reached the previous round, a proposer cannot choose r arbitrarily; it is constrained to rounds open for its verified chain progression.

4.2 Anchor commit and derived skip status

An anchor A_r is *directly committed* when at least $f + 1$ round-$(r{+}1)$ deltas in a node's view reference it. The anchor of a round is *skipped* if it is not directly committed and does not appear in the causal history of a later committed anchor. Status is a pure function of the DAG view, so no separate skip messages are needed.

Assumption 1 (Non-equivocating delivery, A1)**.** *Each honest node accepts at most one delta per (signer, round), for example through a consistent-broadcast layer. A second delta with the same (signer, round) is retained only as equivocation evidence. Without A1, the direct-commit threshold must be raised from $f + 1$ to Q.*

Lemma 2 (Anchor consistency)**.** *Let $N = 3f + 1$ and assume A1. If A_r is directly committed at some honest node, every round-$(r{+}2)$ delta accepted by any honest node has A_r in its causal history.*

Proof. Let S be the set of at least $f + 1$ signers whose round-$(r{+}1)$ deltas reference A_r. A round-$(r{+}2)$ delta d' references round-$(r{+}1)$ deltas from a set T of at least $Q = 2f + 1$ distinct signers. Then $|S \cap T| \geq (f + 1) + (2f + 1) - (3f + 1) = 1$. Take $s \in S \cap T$. By A1 there is a single accepted round-$(r{+}1)$ delta by s, so the delta referenced by d' is the one that references A_r. Hence $A_r \in \text{Hist}(d')$. □

Corollary 1. *Every delta of round $r' \geq r + 2$ has A_r in its causal history (induction on r'). Every committed anchor of round at least $r + 2$ therefore contains A_r, and the skip status of A_r is the same at all honest nodes.*

4.3 Segments, canonical order and future-beacon salting

Committed anchors $A_{R_1}, A_{R_2}, \ldots$ define execution segments

$$\mathrm{Seg}_i = \mathrm{Hist}(A_{R_i}) \setminus \bigcup_{j<i} \mathrm{Hist}(A_{R_j}).$$

To prevent proposers from grinding nonces or payload padding to manipulate tie-breaking, the tie-breaker is salted with a future randomness beacon B_r, a threshold signature over (epoch, r) released only after the inclusion window closes. The sort key of a delta d in segment Seg_i is

$$\mathrm{SortKey}(d) = \big(r(d),\, h(d),\, H(d \,\|\, B_{r(d)})\big).$$

Round numbers are non-decreasing and heights strictly increasing along valid edges, so this order is a valid topological order. Segments execute sequentially, so late-arriving deltas cannot reorder finalized prefixes.

4.4 Beacon timing and inclusion window

Let W be the protocol inclusion window. A delta of round r is *eligible* only if it lies in the causal history of a committed anchor of round $R' \le r + W$; otherwise it is void and must be resubmitted. The salt B_r is released only after the status of every anchor up to round $r + W$ is decided. Execution of round r therefore becomes immutable only after B_r is known, a bounded latency of W rounds plus anchor resolution.

Parameter constraints (recommended, not proven). τ must be at least the post-GST network delay bound Δ, so an honest delta reaches all honest validators before they close their round. W should be at least $f + 3$ rounds: an honest delta of round r is referenced by every honest round-(r+1) delta, and slot rotation can place at most f consecutive Byzantine anchor candidates before an honest one is due. Validating this bound is an open proof obligation.

4.5 Double-spend resolution and state semantics

Conflicting transactions modifying the same state keys are serialized according to the canonical segment order. The first in canonical order executes successfully (`STATUS_SUCCESS`). Any later conflicting transaction sees updated state pre-conditions and terminates with `STATUS_REVERTED` (no-op), giving deterministic rejection without invalidating DAG consensus.

5 Checkpoints, Inclusion and Recovery

5.1 Quorum intersection and checkpoint certificates

With $Q = \lceil (N + f + 1)/2 \rceil$, any two valid quorums intersect in at least $f + 1$ validators, hence in at least one honest validator. A checkpoint certificate is

$$CC(R) = \langle A_R,\ \mathrm{StateRoot}_R,\ \sigma_{\ge Q} \rangle.$$

A validator signs $CC(R)$ only if A_R is directly committed in its view and $\mathrm{StateRoot}_R$ matches its local execution through round R. Because A_R commits to its entire history via hash links, no separate frontier root is required.

5.2 Inclusion and censorship resistance

An honest delta broadcast after GST reaches all honest validators within network delay τ. Every honest round-(r+1) delta then references it, securing at least $N - f \ge f + 1$ links. It enters the causal history of the next committed anchor within the window W. Deltas missing eligibility are void rather than lost, and their payloads can be resubmitted.

5.3 Liveness under partial synchrony

Under partial synchrony with at least Q honest validators online, rounds advance through the round rule and derived anchor commitments alone. An offline slot holder is skipped by DAG derivation, without subjective timeout votes.

5.4 Recovery trust assumption

If conflicting valid checkpoint certificates are detected, nodes persist both and enter `StateHalt`. Resolution requires an out-of-band recovery transaction signed by a pre-designated recovery set $\mathcal{R}$ (fixed in genesis with threshold $t_{\mathcal{R}}$ of M keys). This set is an explicit trust assumption outside the f-Byzantine model.

5.5 Convergence

Theorem 1 (Deterministic convergence)**.** *Let two honest nodes hold the same sequence of committed anchors $A_{R_1}, \ldots, A_{R_k}$, the same beacons B_r for all $r \leq R_k$, and the same initial state S_0. Then they derive identical segments, identical canonical order D_{sorted}, and identical state roots.*

Proof sketch. Each $\text{Hist}(A_{R_i})$ is fixed by hash links, so the segments Seg_i are the same sets at both nodes. SortKey depends only on $r(d)$, $h(d)$, $H(d)$ and $B_{r(d)}$, all of which are determined. Execution is a deterministic function of the ordered sequence and S_0 under the Wasm determinism requirements. □

Agreement on the sequence of committed anchors follows from Lemma 2 together with quorum intersection of checkpoint certificates. A complete proof of that agreement, and of eventual delivery of the same deltas to all honest nodes, is left open.

6 Related Work

The round-and-anchor DAG architecture shares structural foundations with high-throughput DAG consensus protocols such as Narwhal and Bullshark [1, 2] and DAG-Rider [3], which decouple data dissemination from linear ordering. Unlike pure consensus engines, this work integrates:

- **Proof of Invariant (PoI):** incremental sequence-based equivocation guards and inductive structural verification.
- **Regulatory compliance framing:** edge-verified EUDI/eIDAS 2.0 (SD-JWT) and Diia.Signature presentations mapped into transaction execution payloads.
- **Deterministic Wasm execution:** sandboxed state transitions governed by strict canonicalization rules and future-beacon anti-grinding salting.

The round structure, anchor commit rule and quorum-intersection argument follow the Narwhal/Bullshark and DAG-Rider line of work. The additions here are the per-validator sequence-indexed equivocation guard with committed-head pruning, the derived (vote-free) skip status, the inclusion-window and per-round future-beacon scheme for anti-grinding ordering, and the use of committed anchors as checkpoint targets. Unlike those protocols, this design does not assume reliable broadcast, so Lemma 2 states non-equivocating delivery (A1) as an explicit assumption.

7 Conclusion and Limitations

By combining sequence-based equivocation tracking, future-beacon anti-grinding salting, deterministic conflict execution, and committed-anchor checkpoint certificates, Proof of Invariant (PoI) provides an architectural foundation for eventually consistent causal DAG ledgers.

Limitations. The protocol provides conditional finality bounded by checkpoint intervals. It depends on the availability of threshold-signed future beacons and on out-of-band recovery signers for administrative fault resolution. Lemma 2 relies on assumption A1, the bounds on W and τ are not yet proven, and no implementation or evaluation is reported.

References

[1] G. Danezis, L. Kokoris-Kogias, A. Sonnino, A. Spiegelman. Narwhal and Tusk: A DAG-based Mempool and Efficient BFT Consensus. *EuroSys*, 2022.

[2] A. Spiegelman, N. Giridharan, A. Sonnino, L. Kokoris-Kogias. Bullshark: DAG BFT Protocols Made Practical. *ACM CCS*, 2022.

[3] I. Keidar, E. Kokoris-Kogias, O. Naor, A. Spiegelman. All You Need is DAG. *ACM PODC*, 2021.